

TAGORE COLLEGE OF ARTS AND SCIENCE

Approved by the Govt. of Tamil Nadu & Affiliated to the University of Madras
Works Road, Chromepet, Chennai – 600 044



DEPARTMENT OF COMPUTER SCIENCE

Certified to be the Bonafide Record of Practical work done by Mr. /Miss.
..... Reg. No.....of Third Year B.Sc., in the
Department of Computer Science, Shift – I, Tagore College of Arts & Science, Chromepet,
Chennai – 600 044 during the year 2026-2027.

LECTURER IN – CHARGE

HEAD OF THE DEPARTMENT
Department of Computer Science

Submitted for B.Sc., Degree Practical Examination held on at the
Department of Computer Science, Shift – I, Tagore College of Arts and Science,
Chromepet, Chennai –44.

Date :

Examiners

1.

2.

INDEX

S.NO	DATE	NAME OF PROGRAM	P.NO	SIGN
1.		BASIC I/O PROGRAMMING		
2.		CPU SCHEDULING ALGORITHM : FIRST COME FIRST SERVED (FCFS)		
3.		CPU SCHEDULING ALGORITHM : SHORTEST JOB FIRST (SJF)		
4.		CPU SCHEDULING ALGORITHM : ROUND ROBIN (RR)		
5.		CPU SCHEDULING ALGORITHM : PRIORITY		
6.		READER/WRITER PROBLEM USING SEMAPHORE		
7.		BANKER'S ALGORITHM - DEADLOCK AVOIDANCE		
8.		PAGE REPLACEMENT ALGORITHM FIRST IN FIRST OUT (FIFO)		
9.		PAGE REPLACEMENT ALGORITHM : LEAST RECENTLY USED (LRU)		
10. A		MEMORY MANAGEMENT ALGORITHM – FIRST FIT		
10. B		MEMORY MANAGEMENT ALGORITHM – BEST FIT		
10. C		MEMORY MANAGEMENT ALGORITHM – WORST FIT		
11.		INTER PROCESS COMMUNICATION (IPC)		

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

void main()
{
    int i, len, s=0, w=1, c=0, v=0;
    char s1[50], ch;
    clrscr();
    printf("\n\t\tINPUT:");
    printf("\n\t\t-----");
    printf("\n\t\tEnter the Text: ");
    gets(s1);
    len = strlen(s1);

    for(i=0; i<=len-1; i++)
    {
        ch = s1[i];
        switch(ch)
        {
            case 'A': case 'E': case 'I': case 'O': case 'U':
            case 'a': case 'e': case 'i': case 'o': case 'u':
                v++;
                break;
            case ' ':
                s++;
                break;
            default:
                c++;
        }
        if(s1[i] == ' ' && s1[i-1] != ' ')
        {
            w++;
        }
    }
}
```

```
printf("\n\t\tOUTPUT:");  
printf("\n\t\t-----");  
printf("\n\t\tThe number of vowels is: %d", v);  
printf("\n\t\tThe number of consonants is: %d", c);  
printf("\n\t\tThe number of words is: %d", w);  
printf("\n\t\tThe number of white spaces is: %d", s);  
getch();  
}
```

OUTPUT:

 NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

INPUT:

Enter the Text: we studies computer science

OUTPUT:

The number of vowels is: 10

The number of consonats is: 14

The number of words is: 4

The number of white spaces is: 3_

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int bt[20], wt[20], tat[20], i, n;
    float wtavg, tatavg;
    clrscr();
    printf("\nEnter the number of processes--");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\nEnter Burst time of Process%d--", i);
        scanf("%d", &bt[i]);
    }

    wt[0] = wtavg = 0;
    tat[0] = tatavg = bt[0];

    for(i=1; i<n; i++)
    {
        wt[i] = wt[i-1] + bt[i-1];
        tat[i] = tat[i-1] + bt[i];
        wtavg = wtavg + wt[i];
        tatavg = tatavg + tat[i];
    }

    printf("\nPROCESS\tBURST TIME\tWAITING TIME\tTURNAROUND TIME\n");
    for(i=0; i<n; i++)
        printf("\n\t%d\t\t%d\t\t%d\t\t%d", i, bt[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time--%f", wtavg/n);
    printf("\nAverage Turnaround Time--%f", tatavg/n);
    getch();
}
```

OUTPUT:

```
NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Enter the number of processes--3
Enter Burst time of Process0--24
Enter Burst time of Process1--8
Enter Burst time of Process2--12

```

	PROCESS BURST TIME	WAITING TIME	TURNAROUND TIME
p0	24	0	24
p1	8	24	32
p2	12	32	44

```

Average Waiting Time--18.666667
Average Turnaround Time--33.333333_

```

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int p[20], bt[20], wt[20], tat[20], i, k, n, temp;
    float wtavg, tatavg;
    clrscr();
    printf("\nEnter the number of processes--");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p[i] = i;
        printf("\nEnter Burst time for process%d--", i);
        scanf("%d", &bt[i]);
    }
    for(i=0; i<n; i++)
    {
        for(k=i+1; k<n; k++)
        {
            if(bt[i] > bt[k])
            {
                temp = bt[i];
                bt[i] = bt[k];
                bt[k] = temp;

                temp = p[i];
                p[i] = p[k];
                p[k] = temp;
            }
        }
    }
    wt[0] = wtavg = 0;
    tat[0] = tatavg = bt[0];
    for(i=1; i<n; i++)
```

```
{
    wt[i] = wt[i-1] + bt[i-1];
    tat[i] = tat[i-1] + bt[i];
    wtavg = wtavg + wt[i];
    tatavg = tatavg + tat[i];
}
printf("\n\tprocess\tBurst time\tWaiting time\t Turnaround time\n");
for(i=0; i<n; i++)
    printf("\n\tp%d\t\t%d\t\t%d\t\t%d", p[i], bt[i], wt[i], tat[i]);
printf("\nAverage Waiting time--%f", wtavg/n);
printf("\nAverage Turnaround time--%f", tatavg/n);
getch();
}
```

OUTPUT:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Enter the number of processes--3
Enter Burst time for process0--10
Enter Burst time for process1--24
Enter Burst time for process2--12

    process Burst time      Waiting time      Turnaround time
    p0          10           0             10
    p2          12          10             22
    p1          24          22             46
Average Waiting time--10.666667
Average Turnaround time--26.000000
```

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int i, j, n, bu[10], wa[10], tat[10], t, ct[10], max;
    float awt=0, att=0, temp=0;
    clrscr();
    printf("\nEnter the no of process--");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\nEnter Burst time for process%d--", i+1);
        scanf("%d", &bu[i]);
        ct[i] = bu[i];
    }
    printf("\nEnter the size of time slice--");
    scanf("%d", &t);
    max = bu[0];
    for(i=1; i<n; i++)
        if(max < bu[i])
            max = bu[i];
    for(j=0; j<(max/t)+1; j++)
    {
        for(i=0; i<n; i++)
        {
            if(bu[i] != 0)
            {
                if(bu[i] <= t)
                {
                    tat[i] = temp + bu[i];
                    temp = temp + bu[i];
                    bu[i] = 0;
                }
                else
```

```

        {
            bu[i] = bu[i] - t;
            temp = temp + t;
        }
    }
}
for(i=0; i<n; i++)
{
    wa[i] = tat[i] - ct[i];
    att += tat[i];
    awt += wa[i];
}
printf("\nThe Average Turnaround time is--%f", att/n);
printf("\nThe Average Waiting time is--%f", awt/n);
printf("\n\tPROCESS\tBURST TIME\tWAITING TIME\tTURNAROUND TIME\n");
for(i=0; i<n; i++)
    printf("\t%d\t%d\t\t%d\t\t%d\n", i+1, ct[i], wa[i], tat[i]);
getch();
}

```

OUTPUT:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Enter the no of process--3
Enter Burst time for process1--24
Enter Burst time for process2--12
Enter Burst time for process3--8
Enter the size of time slice--2

The Average Turaround time is--33.33333
The Average Waiting time is--18.666667
```

PROCESS	BURST TIME	WAITING TIME	TURNAROUND TIME
1	24	20	44
2	12	20	32
3	8	16	24

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int p[20], bt[20], pri[20], wt[20], tat[20], i, n, k, temp;
    float wtavg, tatavg;
    clrscr();
    printf("\nEnter the number of processes--");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        p[i] = i;
        printf("\nEnter the Burst Time & Priority of process%d--", i);
        scanf("%d%d", &bt[i], &pri[i]);
    }
    for(i=0; i<n; i++)
    {
        for(k=i+1; k<n; k++)
        {
            if(pri[i] < pri[k]) /* higher number = higher priority (matches lab manual) */
            {
                temp = p[i];
                p[i] = p[k];
                p[k] = temp;

                temp = bt[i];
                bt[i] = bt[k];
                bt[k] = temp;

                temp = pri[i];
                pri[i] = pri[k];
                pri[k] = temp;
            }
        }
    }
}
```

```

    }
    wtavg = wt[0] = 0;
    tatavg = tat[0] = bt[0];
    for(i=1; i<n; i++)
    {
        wt[i] = wt[i-1] + bt[i-1];
        tat[i] = tat[i-1] + bt[i];
        wtavg = wtavg + wt[i];
        tatavg = tatavg + tat[i];
    }
    printf("\nPROCESS\t\tPRIORITY\tBURST TIME\tWAITING TIME\tTURNAROUND TIME");
    for(i=0; i<n; i++)
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", p[i], pri[i], bt[i], wt[i], tat[i]);
    printf("\nAverage Waiting Time is--%f", wtavg/n);
    printf("\nAverage Turnaround Time is--%f", tatavg/n);
    getch();
}

```

OUTPUT:

```
NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Enter the number of processes--5
Enter the Burst Time & Priority of process0--10 3
Enter the Burst Time & Priority of process1--1 1
Enter the Burst Time & Priority of process2--2 4
Enter the Burst Time & Priority of process3--1 5
Enter the Burst Time & Priority of process4--5 2
PROCESS          PRIORITY          BURST TIME          WAITING TIME          TURNROUND TIME
1                 1                 1                 0                 1
4                 2                 5                 1                 6
0                 3                 10                6                 16
2                 4                 2                 16                18
3                 5                 1                 18                19
Average Waiting Time is--8.200000
Average Turnaround Time is--12.000000_
```

SOURCE CODE :

```
#include<semaphore.h>
#include<stdio.h>
#include<stdlib.h>

sem_t x,y;
pthread_t tid;
pthread_t
writerthreads[100],readerthreads[100];
int readercount;
void * reader (void * param)
{
sem_wait(&w);
readercount++;
if(readercount==1)
sem_wait(&y);
sem_post(&x);
printf("\n%d reader is inside", readercount);
sem_wait(&x);
readercount--;
if(readercount==0)
{
sem_post(&y);
}
sem_post(&x);
printf("\n%d Reader is leaving", readercount+1);
}
void * writer(void * param)
```

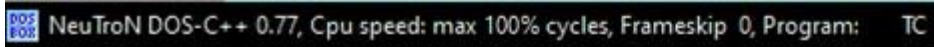
```

{
printf("\nWriter is trying to enter");
sem_wait(&y);
printf("\nWriter has entered");
sem_post(&y);
printf("\nWriter is leaving");
}

int main()
{
inbt n2,i;
printf("Enter the number of readers:");
scanf("%d",&n2);
int n1[n2];
sem_init(&x,0,1);
sem_init(&y,0,1);
for(i=0;i<n2;i++)
pthread_create(&writerthreads[i],NULL,reader,NULL);
pthread_create(&readerthreads[i],NULL,writer,NULL);
}
for(i=0;i<n2;i++)
{
pthread_join(writerthreads[i],NULL);
pthread_join(readerthreads[i],NULL);
}
}

```

OUTPUT:

A black horizontal bar with white text. On the left is the DOSBox logo. The text reads: "NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC".

DOSBOX NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

The output goes in the form of

reader is inside

reader is leaving

reader is inside

reader is leaving

writer is trying to enter

writer has entered

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
#include<string.h>

void main()
{
    int alloc[10][10], max[10][10];
    int avail[10], work[10], total[10];
    int i, j, n, need[10][10];
    int m, k;
    int count=0, c=0, progress;
    char finish[10];
    clrscr();

    printf("Enter the no. of processes and resources:");
    scanf("%d%d", &n, &m);
    for(i=0; i<n; i++)        /* fixed: was i<=n, wrote one byte past the array */
        finish[i] = 'n';

    printf("Enter the claim matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter the allocation matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Resource vector:");
    for(i=0; i<m; i++)
        scanf("%d", &total[i]);

    for(i=0; i<m; i++)
        avail[i] = 0;
```

```

for(i=0; i<n; i++)
    for(j=0; j<m; j++)
        avail[j] += alloc[i][j];

for(i=0; i<m; i++)
    work[i] = avail[i];

for(j=0; j<m; j++)
    work[j] = total[j] - work[j];

for(i=0; i<n; i++)
    for(j=0; j<m; j++)
        need[i][j] = max[i][j] - alloc[i][j];

```

A:

```

progress = 0;          /* new: tracks whether any process ran this pass */
for(i=0; i<n; i++)
{
    c = 0;
    for(j=0; j<m; j++)
        if((need[i][j] <= work[j]) && (finish[i] == 'n'))
            c++;

    if(c == m && finish[i] == 'n')
    {
        printf("All the resources can be allocated to process %d", i+1);
        printf("\n\nAvailable resources are:");
        for(k=0; k<m; k++)
        {
            work[k] += alloc[i][k];
            printf("%d ", work[k]);
        }
        printf("\n");
        finish[i] = 'y';
        printf("\nProcess %d executed?:%c\n", i+1, finish[i]);
        count++;
        progress = 1;    /* new: at least one process finished this pass */
    }
}

```

```

}

if(count != n)
{
    if(progress == 1)
        goto A;          /* still making progress, do another pass */
    else
    {
        /* new: no process could run this pass -> state is genuinely unsafe */
        printf("\nSystem is NOT in safe mode");
        printf("\nThe given state is an UNSAFE state (deadlock possible)");
        getch();
        return;
    }
}

printf("\nSystem is in safe mode");
printf("\nThe given state is safe state");
getch();
}

```

OUTPUT:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Enter the no. of processes and resources:4 3
Enter the claim matrix:
3 2 2
6 1 3
3 1 4
4 2 2
Enter the allocation matrix:
1 0 0
6 1 2
2 1 1
```

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
0 0 2
Resource vector:9 3 6
All the resources can be allocated to process 2

Available resources are:623

Process 2 executed?:y
All the resources can be allocated to process 3

Available resources are:834

Process 3 executed?:y
All the resources can be allocated to process 4

Available resources are:836

Process 4 executed?:y
All the resources can be allocated to process 1

Available resources are:936

Process 1 executed?:y

System is in safe mode
The given state is safe state_
```

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

int fr[3];
void display();

void main()
{
    int i, j, page[12] = {2,3,2,1,5,2,4,5,3,2,5,2};
    int flag1=0, flag2=0, pf=0, frsize=3, top=0;
    clrscr();

    for(i=0; i<3; i++)
    {
        fr[i] = -1;
    }

    for(j=0; j<12; j++)
    {
        flag1 = 0;
        flag2 = 0;
        for(i=0; i<3; i++)
        {
            if(fr[i] == page[j])
            {
                flag1 = 1;
                flag2 = 1;
                break;
            }
        }
        if(flag1 == 0)
        {
            for(i=0; i<frsize; i++)
            {
                if(fr[i] == -1)
```

```

        {
            fr[i] = page[j];
            flag2 = 1;
            break;
        }
    }
}
if(flag2 == 0)
{
    fr[top] = page[j];
    top++;
    pf++;
    if(top >= frsize)
        top = 0;
}
display();
}
printf("\nNumber of page faults:%d", pf+frsize);
getch();
}

void display()
{
    int i;
    printf("\n");
    for(i=0; i<3; i++)
        printf("%d\t", fr[i]);
}

```

OUTPUT:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

2      -1      -1
2      3       -1
2      3       -1
2      3        1
5      3        1
5      2        1
5      2        4
5      2        4
3      2        4
3      2        4
3      5        4
3      5        2      Number of page faults:9
```

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>

int fr[3];
void display();

void main()
{
    int p[12] = {2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2}, i, j, fs[3];
    int index, k, l, flag1 = 0, flag2 = 0, pf = 0, frsize = 3;
    clrscr();
    for(i=0; i<3; i++)
    {
        fr[i] = -1;
    }
    for(j=0; j<12; j++)
    {
        flag1 = 0;
        flag2 = 0;
        for(i=0; i<3; i++)
        {
            if(fr[i] == p[j])
            {
                flag1 = 1;
                flag2 = 1;
                break;
            }
        }
        if(flag1 == 0)
        {
            for(i=0; i<3; i++)
            {
                if(fr[i] == -1)
                {
                    fr[i] = p[j];
```

```

        flag2 = 1;
        break;
    }
}
}
if(flag2 == 0)
{
    for(i=0; i<3; i++)
        fs[i] = 0;
    for(k=j-1, l=1; l<=frsize-1; l++, k--)
    {
        for(i=0; i<3; i++)
        {
            if(fr[i] == p[k])
                fs[i] = 1;
        }
    }
    for(i=0; i<3; i++)
    {
        if(fs[i] == 0)
            index = i;
    }
    fr[index] = p[j];
    pf++;
}
display();
}

printf("\n No of page faults:%d", pf + frsize);
getch();
}

void display()
{
    int i;
    printf("\n");
    for(i=0; i<3; i++)
        printf("\t%d", fr[i]);
}

```

OUTPUT:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
2      -1     -1
2      3      -1
2      3      -1
2      3      1
2      5      1
2      5      1
2      5      4
2      5      4
3      5      4
3      5      2
3      5      2
3      5      2
no of page faults:7
```

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
#define max 25

void main()
{
    int frag[max], b[max], f[max], i, j, nb, nf, temp;
    static int bf[max], ff[max];
    clrscr();
    printf("\nMemory Management Scheme: First Fit");
    printf("\nEnter the number of blocks:");
    scanf("%d", &nb);
    printf("Enter the number of files:");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1; i<=nb; i++)
    {
        printf("Block %d:", i);
        scanf("%d", &b[i]);
    }
    printf("Enter the size of the files:-\n");
    for(i=1; i<=nf; i++)
    {
        printf("File %d:", i);
        scanf("%d", &f[i]);
    }
    for(i=1; i<=nf; i++)
    {
        for(j=1; j<=nb; j++)
        {
            if(bf[j] != 1)
            {
                temp = b[j] - f[i];
                if(temp >= 0)
                {
```

```

        ff[i] = j;
        break;
    }
}
}
if(ff[i] == 0)
{
    printf("\nFile %d: No suitable block found!", i);
    frag[i] = -1;
    continue;
}
frag[i] = temp;
bf[ff[i]] = 1;
}
printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragment");
for(i=1; i<=nf; i++)
{
    if(ff[i] == 0)
        printf("\n%d\t\t%d\t\tNot Allocated", i, f[i]);
    else
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]);
}
getch();
}

```

OUTPUT:

Memory Management Scheme: First Fit

Enter the number of blocks:3

Enter the number of files:2

Enter the size of the blocks:-

Block 1:5

Block 2:2

Block 3:7

Enter the size of the files:-

File 1:1

File 2:4

File_no:	File_size:	Block_no:	Block_size:	Fragment
1	1	1	5	4
2	4	3	7	3

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
#define max 25

void main()
{
    int frag[max], b[max], f[max], i, j, nb, nf, temp, lowest;
    static int bf[max], ff[max];
    clrscr();
    printf("\nMemory Management Scheme: Best Fit");
    printf("\nEnter the number of blocks:");
    scanf("%d", &nb);
    printf("Enter the number of files:");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1; i<=nb; i++)
    {
        printf("Block %d:", i);
        scanf("%d", &b[i]);
    }
    printf("Enter the size of the files:-\n");
    for(i=1; i<=nf; i++)
    {
        printf("File %d:", i);
        scanf("%d", &f[i]);
    }
    for(i=1; i<=nf; i++)
    {
        lowest = 10000;
        for(j=1; j<=nb; j++)
        {
            if(bf[j] != 1)
            {
                temp = b[j] - f[i];
                if(temp >= 0)
```

```

        {
            if(lowest > temp)
            {
                ff[i] = j;
                lowest = temp;
            }
        }
    }
}
if(ff[i] == 0)
{
    printf("\nFile %d: No suitable block found!", i);
    frag[i] = -1;
    continue;
}
frag[i] = lowest;
bf[ff[i]] = 1;
}
printf("\nFile_no\tFile_size\tBlock_No\tBlock_size\tFragment");
for(i=1; i<=nf; i++)
{
    if(ff[i] == 0)
        printf("\n%d\t\t%d\t\tNot Allocated", i, f[i]);
    else
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]);
}
getch();
}

```

OUTPUT:

```
Memory Management Scheme: Best Fit
Enter the number of blocks:3
Enter the number of files:2
```

```
Enter the size of the blocks:-
Block 1:5
Block 2:2
Block 3:7
Enter the size of the files:-
File 1:1
File 2:4
```

File_no	File_size	Block_No	Block_size	Fragment
1	1	2	2	1
2	4	1	5	1

SOURCE CODE :

```
#include<stdio.h>
#include<conio.h>
#define max 25

void main()
{
    int frag[max], b[max], f[max], i, j, nb, nf, temp, highest;
    static int bf[max], ff[max];
    clrscr();
    printf("\nMemory Management Scheme: Worst Fit");
    printf("\nEnter the number of blocks:");
    scanf("%d", &nb);
    printf("Enter the number of files:");
    scanf("%d", &nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1; i<=nb; i++)
    {
        printf("Block %d:", i);
        scanf("%d", &b[i]);
    }
    printf("Enter the size of the files:-\n");
    for(i=1; i<=nf; i++)
    {
        printf("File %d:", i);
        scanf("%d", &f[i]);
    }
    for(i=1; i<=nf; i++)
    {
        highest = -1;
        for(j=1; j<=nb; j++)
        {
            if(bf[j] != 1)
            {
                temp = b[j] - f[i];
                if(temp >= 0)
```

```

        {
            if(highest < temp)
            {
                ff[i] = j;
                highest = temp;
            }
        }
    }
}
if(ff[i] == 0)
{
    printf("\nFile %d: No suitable block found!", i);
    frag[i] = -1;
    continue;
}
frag[i] = highest;
bf[ff[i]] = 1;
}
printf("\nFile_no:\tFile_Size:\tBlock_no:\tBlock_size:\tFragment");
for(i=1; i<=nf; i++)
{
    if(ff[i] == 0)
        printf("\n%d\t\t%d\t\tNot Allocated", i, f[i]);
    else
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d", i, f[i], ff[i], b[ff[i]], frag[i]);
}
getch();
}

```

OUTPUT:

Memory Management Scheme: Worst Fit

Enter the number of blocks:3

Enter the number of files:2

Enter the size of the blocks:-

Block 1:5

Block 2:2

Block 3:7

Enter the size of the files:-

File 1:1

File 2:4

File_no:	File_Size:	Block_no:	Block_size:	Fragment
1	1	3	7	6
2	4	1	5	1

SOURCE CODE :

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
main()
{
int p1[2],p2[2],p3[2],p4[2];
int i,j=0,k=0;l=0;
char r[10],s[10],t[10],u[10];
printf("\tPROCESS 1 ENTER THE STRING");
scanf("%s",r);
pipe(p1);
pipe(p2);
write(p1[1],r,sizeof(r));
write(p2[1],r,sizeof(r));
if(a==0)
{
printf("\n\t PROCESS 2: it splite the given string\n");
read(p1[0],r,sizeof(r));
int n=strlen(r);
for(i=0;i<n/2;i++)
{
t[j++]=r[i];
}
pipe(p3);
pipe(p4);
write(p3[1],s,sizeof(s));
```

```

write(p4[1],t,sizeof(t));
int b=fork();
if(b==0)
{
printf("p4 %d\t", getpid());
printf("p2 %d\n", getppid());
read(p3[0],s,sizeof(s));
printf("\t PROCESS 4: sub string \t %d\t",s);
printf("no of char=%d\n",strlen(s));
}
else{
int c=fork();
if(c==0)
{
printf("p5 %d\t",getpid());
printf("p2 %d\n",getppid());
read(p4[0],t,sizeof(t));
printf("\t PROCESS 5: sub string \t %s\t",t);
else
{
wait();
printf("p2 %d\t",getpid());
printf("p1 %d\n\n",getppid());
}
}
}
else
{

```

```

wait();
int d=fork();
if(d==0)
{
printf("p3 %d\t",getpid());
printf("p1 %d\n",getppid());
read(p2[0],r,sizeof(r));
for(i=strlen(r)-1;i>=0;i--)
{
u[l++]=r[i];
}
for(i=0;i<strlen(r);i++)
{
if(u[i]==r[i])
k++;
else continue;
}
if(k==strlen(r))
printf("\tPROCESS 3: the given string is palindrome\n");
else
{
printf("p1 %d\t",getpid());
printf("kernal %d\t\n",getppid());
}
}
}
}

```

OUTPUT:

```
NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Process 1: enter the string
ARUN

Process 2: it split eth string
P4 8137 p2 8136

Process 3: sub string a r no of char=2
P5 8138 p2 8136

Process 4: sub string u n no of char=2
P2 8136 P1 8132

Process 3: the given string string is not
palindrome
```

```
NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

Process 1: enter the string MADAM

Process 2: it splits the string
P4 8137 P2 8136

Process 4: sub string m a no of char=2
P5 8138 P2 8136

Process 5: sub string dam no of char=3
P2 8136 P1 8132
P3 8139 P1 8132

Process 3: the given string is
palindrome
```